Heart Disease Prediction MLOps Project Report

End-to-End Machine Learning Solution & Production Pipeline

1. Title Page

Project Title: Heart Disease Prediction MLOps Project

BITS ID: 2024ad05313

Submitted By: Ankur Sharma

Course / Program: BITS Pilani / MLOps Project

Submission Date: 12 July 2026

2. Project Overview

This project demonstrates an end-to-end machine learning solution for predicting heart disease risk. The system combines data preprocessing, model training, a Flask-based web application, and deployment automation using Docker, GitHub Actions, and Azure.

The primary objective of this work is to move beyond notebook-based experimentation and implement a deployable, production-style MLOps workflow that can serve predictions through a user-friendly interface.

Key Deliverables

- A trained heart disease prediction model
- A Flask web application for single and batch predictions
- Automated CI/CD pipeline for testing and deployment
- Dockerized application for container-based deployment
- Azure deployment setup with health checks and web app hosting

3. EDA Findings

Exploratory Data Analysis (EDA) was performed to understand the structure and characteristics of the heart disease dataset before model training.

Important Observations

- The dataset contains multiple medical attributes such as age, sex, cholesterol, blood pressure, and resting ECG results.
- Several features appear to be strongly correlated with heart disease risk.
- The target variable is categorical and suitable for classification models.
- Feature scaling and preprocessing are necessary to improve model stability.

EDA Summary

- Numerical attributes were analyzed for distribution and variance.
- Categorical variables were reviewed for their contribution to the target outcome.
- Missing values and inconsistent values were handled during preprocessing.

> *EDA helped in selecting appropriate preprocessing strategies and building a robust training pipeline.*

4. Model Comparison

Several supervised classification algorithms were compared to identify the best-performing model for heart disease prediction.

Models Evaluated

- Logistic Regression
- Random Forest Classifier
- Support Vector Machine (SVM)

Comparison Criteria

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

Result Summary

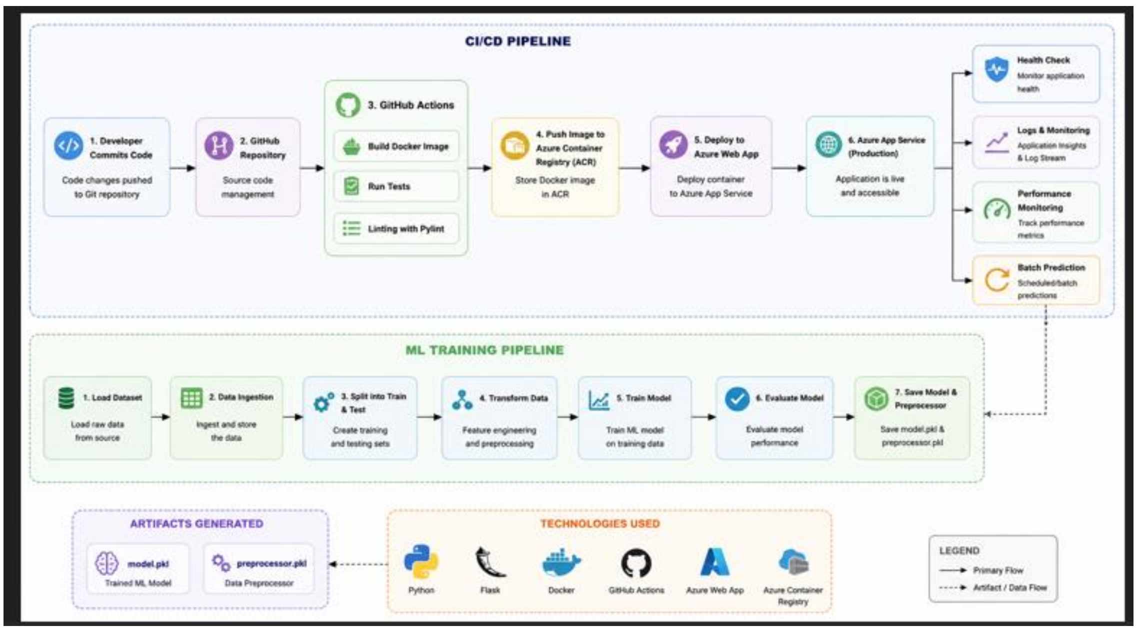
The best-performing model was selected based on the highest overall evaluation metrics and saved as a serialized artifact for prediction use.

The chosen model is stored in the artifacts directory and loaded during inference through the Flask prediction pipeline.

5. Architecture Diagram

[Architecture Flowchart Textual Representation]

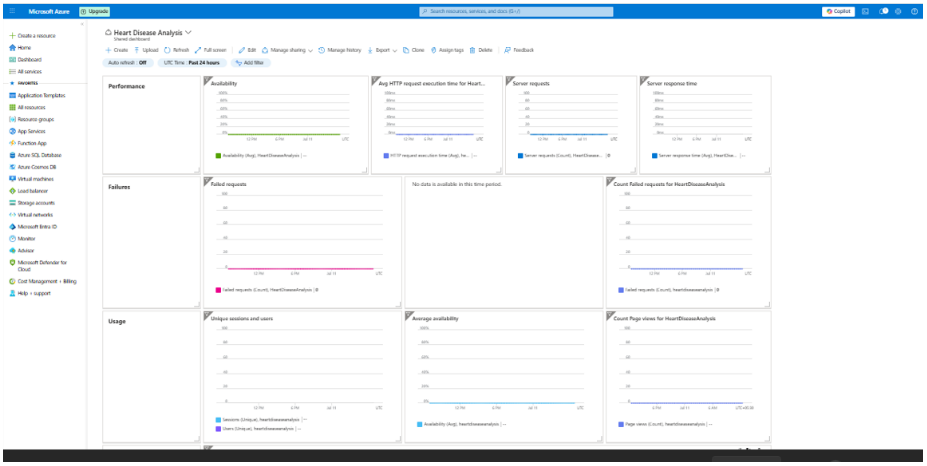
```
User -> Flask Web App -> Prediction API -> Trained ML Model -> Prediction Result

Training Pipeline:
Dataset -> Data Ingestion -> Data Transformation -> Model Training -> Model Evaluation
-> Artifacts (model.pkl)

DevOps Pipeline:
GitHub Repo -> GitHub Actions (flake8, pytest, Docker Build) -> Azure Container
Registry -> Azure Web App
```



6. MLflow and Monitoring Screenshots

MLflow and monitoring tools were used to track experiments, model metrics, and system behavior during development.

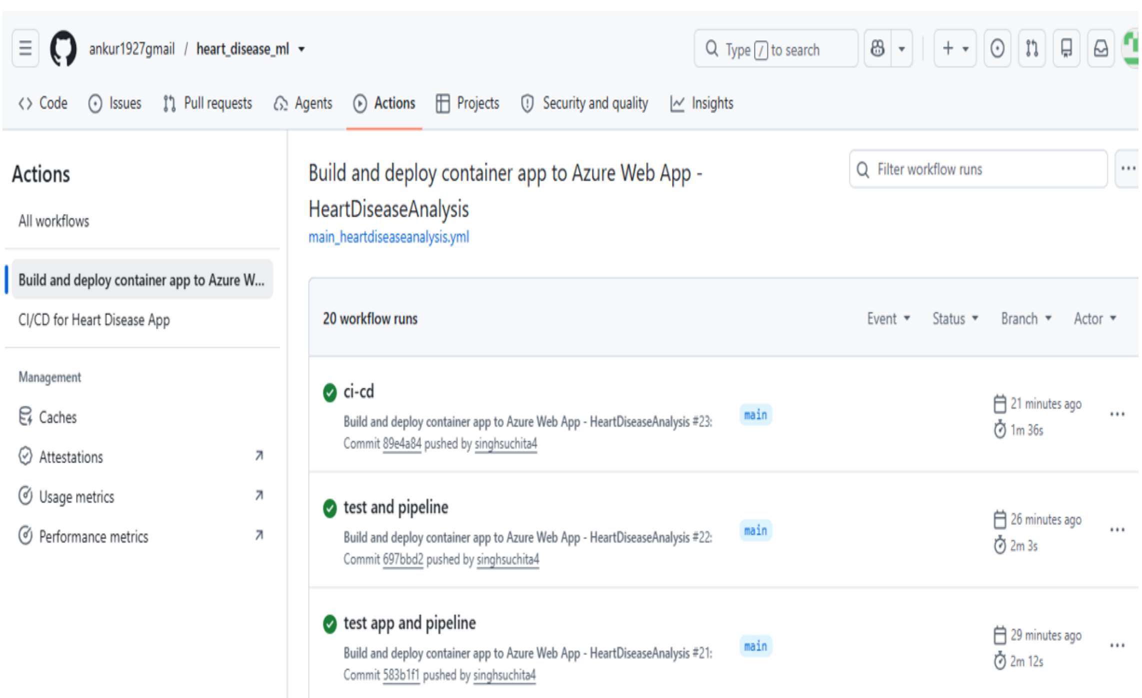
Screenshot 1: Monitoring Dashboard



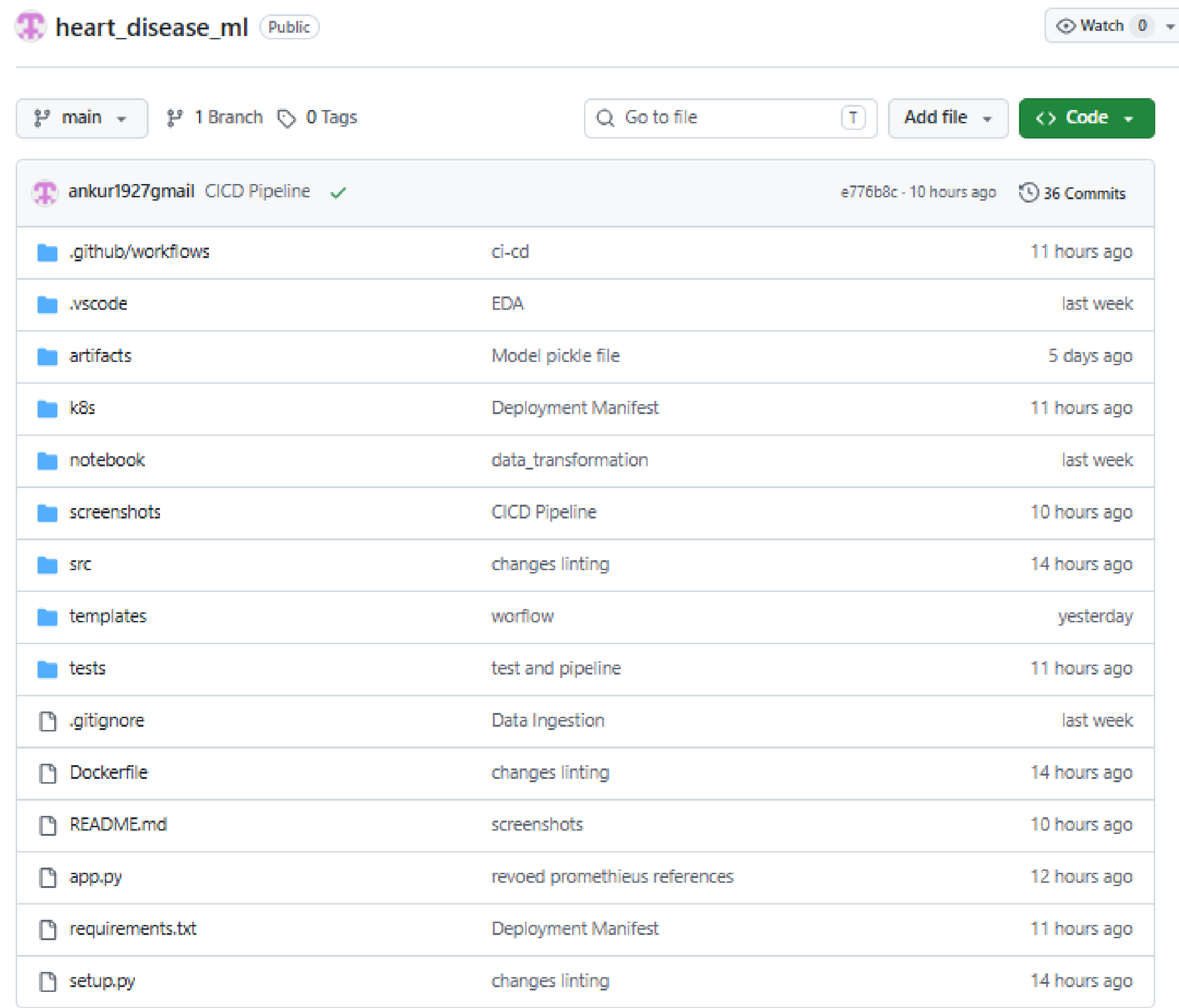
7. CI/CD Screenshots

GitHub Actions was used to automate linting, testing, and deployment.

Screenshot 2: GitHub Actions Workflow



Screenshot 3: GitHub Repository Main View

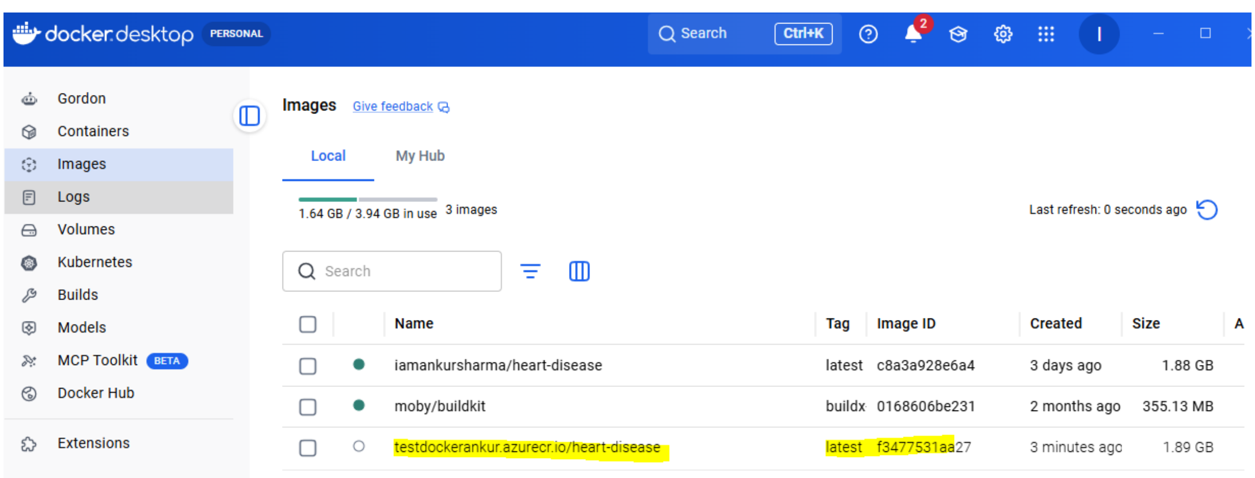


Screenshot 4: CI/CD Pipeline Summary

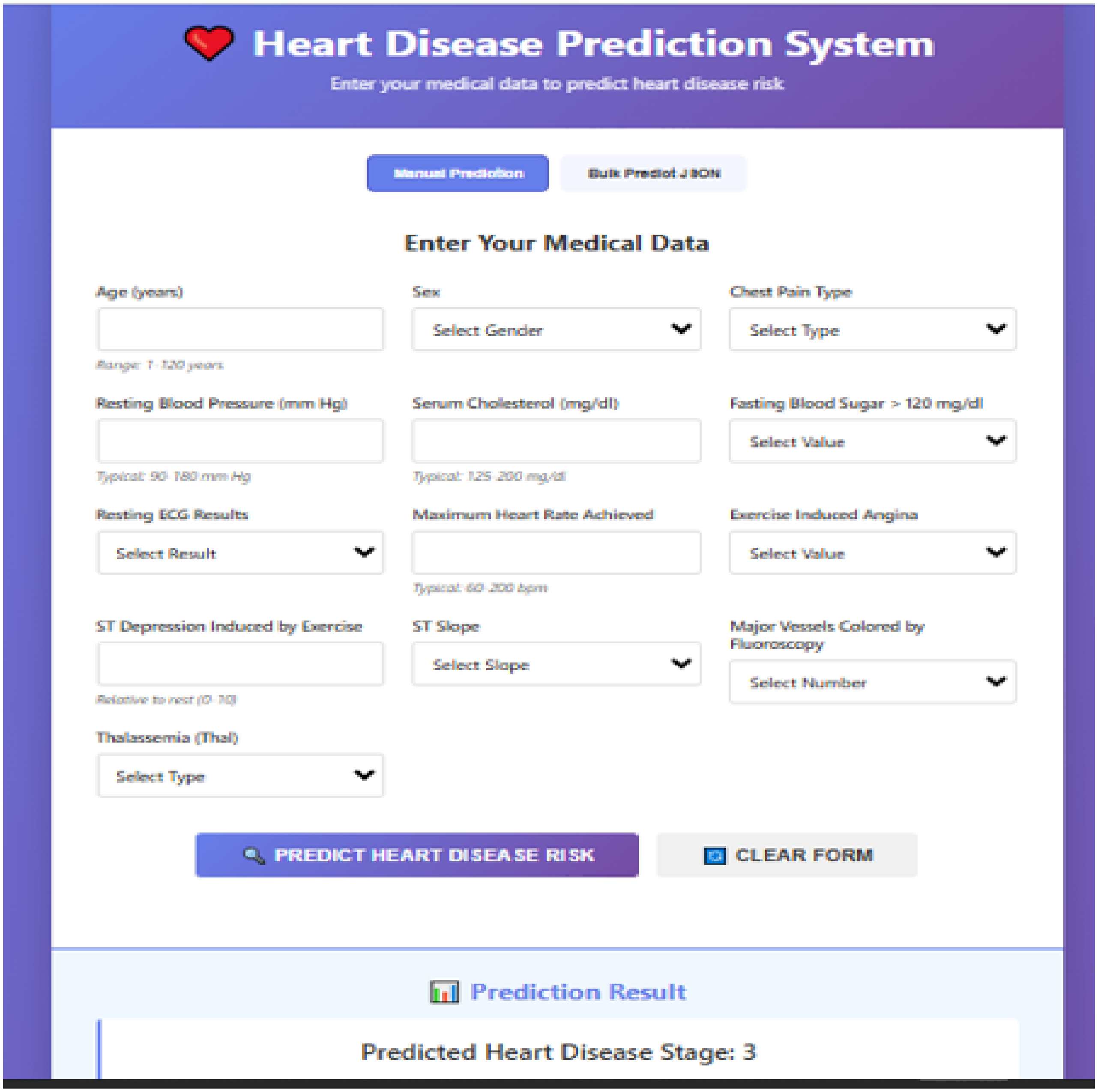
8. Deployment Screenshots

The application was containerized and deployed to Azure.

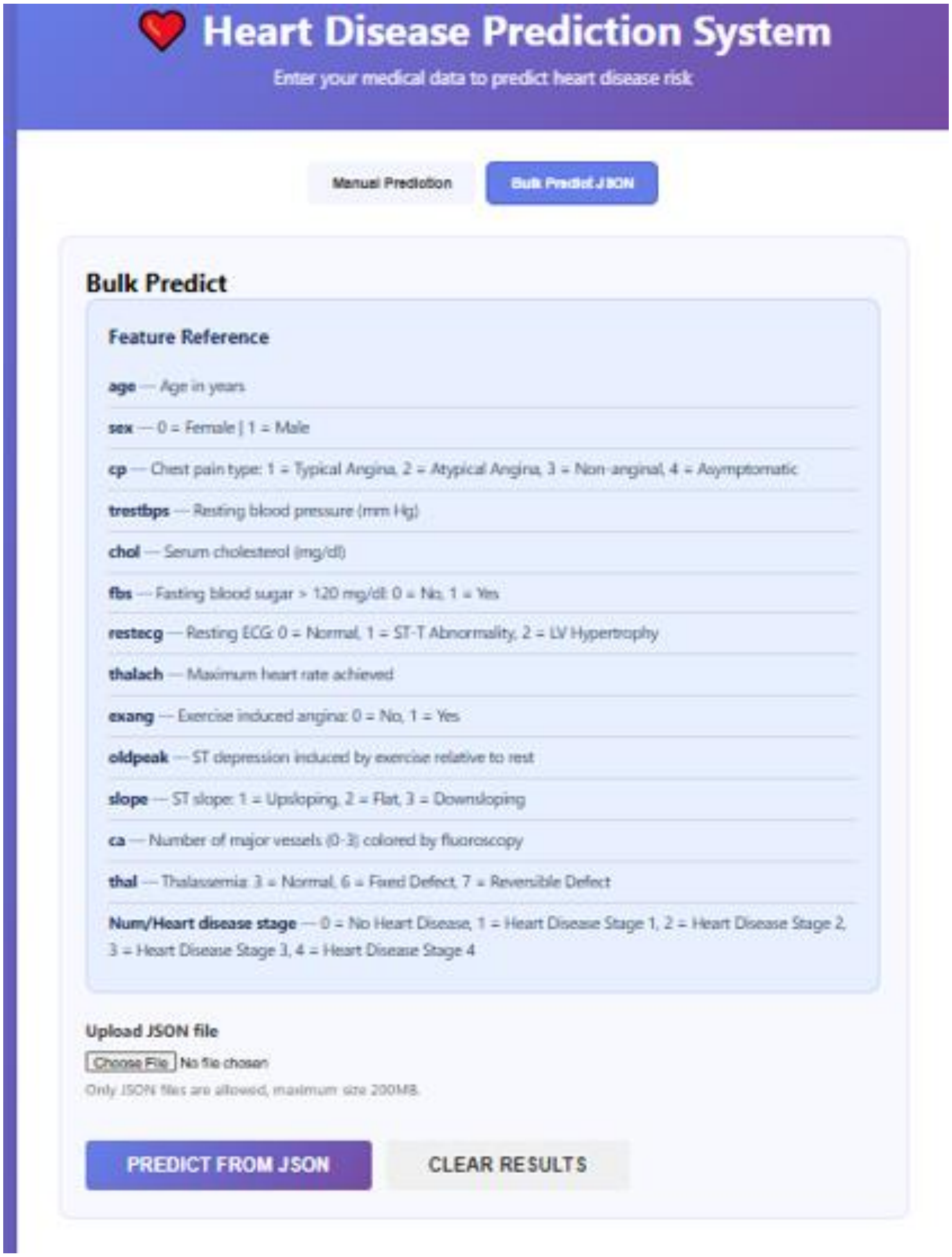
Screenshot 5: Azure Container Image



Screenshot 6: Application UI - Prediction Interface



Screenshot 7: Application UI - Bulk Prediction View



Screenshot 8: Project Structure Reference

```
heart_disease_mlops/
├── .github/
│   └── workflows/
│       ├── ci-cd.yml
│       └── main_heartdiseaseanalysis.yml
├── app.py
├── Dockerfile
├── requirements.txt
├── setup.py
├── README.md
├── artifacts/
│   ├── data.csv
│   ├── test.csv
│   ├── train.csv
│   ├── model.pkl
│   └── preprocessor.pkl
├── k8s/
│   ├── deployment.yaml
│   ├── ingress.yaml
│   └── hpa.yaml
├── logs/
├── notebook/
│   ├── EDA_HEART_DISEASE.ipynb
│   ├── MODEL_TRAINING.ipynb
│   └── data/
│       ├── heart_disease_uci.csv
│       └── preprocessed.csv
├── screenshots/
├── src/
│   ├── __init__.py
│   ├── exception.py
│   ├── logger.py
│   ├── utils.py
│   ├── components/
│   │   ├── __init__.py
│   │   ├── data_ingestion.py
│   │   ├── data_transformation.py
│   │   └── model_trainer.py
│   └── pipeline/
│       ├── __init__.py
│       ├── predict_pipeline.py
│       └── train_pipeline.py
├── templates/
│   ├── home.html
│   └── index.html
├── tests/
│   └── test_app_and_pipeline.py
└── venv312/
```

These deployment and UI screenshots provide visual evidence of the completed solution and its hosted application experience.

9. Setup Instructions

Prerequisites

- Python 3.11
- Docker
- Git
- Azure account (for deployment)

Clone the Repository

```
git clone <repository-url>
cd heart_disease_mlops
```

Create a Virtual Environment

```
python -m venv venv
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Run the Application Locally

```
python app.py
```

The application will run at:

```
http://localhost:8000
```

Run Tests

```
pytest -q
```

Run Lint Checks

```
python -m flake8 . --exclude=.git,venv,venv312,.venv,__pycache__
```

Build Docker Image

```
docker build -t heart-disease .
```

10. Repository Link

Repository: https://github.com/ankur1927gmail/heart_disease_ml

API Link: https://heartdiseaseanalysis-c6bbb8fgbdhdajby.centralus-01.azurewebsites.net

11. Conclusion

This project successfully demonstrates an end-to-end MLOps workflow for heart disease prediction. It covers data analysis, model training, deployment, automation, and cloud delivery in a structured and practical manner.

The solution provides a strong foundation for future enhancement such as monitoring, retraining, model versioning, and production-grade observability.